

fastEmbedR: Supplementary Material

Moussa Kassim^{1,2}

MOUSSA.KASSIM@ICGEB.ORG

Martin Ocharo^{1,2}

MARTIN.OCHARO@ICGEB.ORG

Alessia Vignoli^{3,4}

VIGNOLI@CERM.UNIFI.IT

Leonardo Tenori^{3,4}

TENORI@CERM.UNIFI.IT

Stefano Cacciatore^{1,2}

STEFANO.CACCIATORE@ICGEB.ORG

¹*Bioinformatics Unit, International Center for Genetic Engineering and Biotechnology, Cape Town 7925, South Africa*

²*Department of Integrative Biomedical Sciences, Institute of Infectious Disease & Molecular Medicine (IDM), University of Cape Town, Cape Town 7925, South Africa*

³*Department of Chemistry “Ugo Schiff”, University of Florence, Sesto Fiorentino, Italy*

⁴*Magnetic Resonance Center (CERM), University of Florence, Sesto Fiorentino, Italy*

Editor: To be assigned

S1 Software Contribution and Dependency Boundary

Table S1 separates package-owned code from optional external primitives. This distinction is important: **fastEmbedR** is not a thin wrapper around an existing embedding executable. The R API, input validation, graph/affinity mathematics, optimizer orchestration, transformations, returned objects, and backend reporting are implemented in the package. Optional libraries are linked only where they provide a platform primitive that would be inappropriate to reimplement, such as cuFFT or the cuVS CUDA search API.

The package source includes exact copyright notices and upstream licenses for code that was distilled from permissive projects. It does not vendor Python openTSNE, RAPIDS cuML, uwot, Rtsne, or t-SNE-CUDA. The optional CUDA build links to system installations rather than redistributing those binaries.

Table S1: Ownership of the principal computational stages.

Stage	Implemented in fastEmbedR	Optional primitive or design reference
R interface	One-call and KNN-input APIs; validation; meta-data; plotting and timing objects	Rcpp interface only
Neighbor search	CPU HNSW; Metal exact/IVF orchestration; CUDA buffer ownership and routing	FAISS design/code under MIT; installed FAISS GPU or cuVS C API for CUDA; Faiss-mlx as Apache-2.0 design reference
PCA initialization	Centering/scaling, RSVD orchestration, Metal block-subspace TSVD, t-SNE rescaling	fastPLS-style RSVD; MPS matrix products; RAFT TSVD on CUDA
UMAP graph	Smooth-KNN bandwidth, fuzzy union or binary symmetrization, CSR construction, edge schedule	UMAP paper and uwot used as behavioral references
UMAP optimization	CPU, Metal, and CUDA samplers, RNG, learning-rate decay, attractive/repulsive updates	No Python or cuML call
t-SNE affinities	Per-row perplexity search, sparse symmetrization and normalization	Rtsne/openTSNE behavior used for validation
t-SNE optimization	CPU grid FFT, Metal Stockham/Cooley–Tukey kernels, CUDA sparse/grid kernels and state management	cuFFT on CUDA; Apple-SiliconFFT as MIT design reference
New samples	Landmark selection, KNN interpolation, local affine correction, fixed-reference refinement	openTSNE reference-embedding formulation

S2 Compiler and Native Build Contract

Table S2 distinguishes package-owned compilation requirements from optimization flags inherited from the active R installation. The portable core does not replace `CXX17FLAGS`; consequently, all methods compared on one machine were installed in the same R/toolchain environment. This prevents a change in compiler, standard library, BLAS, or CPU target from being reported as an embedding-algorithm speedup.

Table S2: Native compiler contract.

Layer	Required package configuration	Optimization and portability policy
CPU C++	C++17 and <code>-pthread</code> ; compiler and linker selected by <code>R CMD config</code>	Inherit R's release flags; no package-global OpenMP, <code>-march=native</code> , <code>-ffast-math</code> , or <code>-O3</code>
Apple Metal	Objective-C++ bridge; Foundation, Metal, MPS, and MPSGraph frameworks; Xcode SDK	Runtime Metal compilation targets the installed Apple GPU; fast arithmetic is local to the float32 KNN shader, not a global host flag
NVIDIA CUDA	NVCC C++17, extended lambdas, relaxed <code>constexpr</code> , and position-independent host code	<code>FASTEMBEDR_CUDA_ARCH</code> defines package fat-binary targets; <code>CUDAHOSTCXX</code> selects an R-ABI-compatible compiler
CUDA libraries	CUDA runtime, cuFFT, cuBLAS/cuSOLVER, FAISS GPU, cuVS; RAFT/RMM/CCCL for TSVD	Every linked library must independently contain code for each deployment GPU

For example, `FASTEMBEDR_CUDA_ARCH="75 89"` compiles `fastEmbedR` kernels for both a T4 and an L40S. It does not add `sm_89` kernels to a `cuVS` or `FAISS` binary built only for `sm_75`; such a mismatch produces `cudaErrorNoKernelImageForDevice`. Similarly, compatible CUDA, CCCL, RAFT, RMM, and `libnvJitLink` releases must be resolved from one runtime prefix. Explicit CUDA build switches are strict and configuration stops when requested headers or libraries are absent.

The rationale for conservative host flags was checked empirically. The same HNSW source performed differently under the validated Conda/R GCC toolchain and a generic system R/GCC installation on the same remote machine. An additional `-O3 -march=native` build was slower on MNIST70k and was rejected. Retained HNSW construction improvements instead reuse visit tables, bounded heaps, and worker storage, parallelize reciprocal-row updates, and release construction-only distance caches before query. They preserved exact returned indices and distances on MNIST70k, Fashion-MNIST70k, USPS, and MetRef while reducing four-thread MNIST70k construction from 23.730 to 17.345 seconds and complete KNN time from 27.653 to 21.288 seconds.

Each frozen benchmark manifest records `CC`, `CFLAGS`, `CXX17`, `CXX17FLAGS`, `CPPFLAGS`, `LDFLAGS`, generated `src/Makevars`, compiler versions, Xcode/Metal or CUDA/NVCC versions, `CUDAHOSTCXX`, CUDA architecture flags, and the relevant linked-library versions. Complete source-install commands and validation probes are maintained in the repository build guide.

S3 Public R API

The primary data-matrix calls are:

```
fit_t <- opentsne(x, perplexity = 15, backend = "cpu")
fit_u <- umap(x, n_neighbors = 30, graph_mode = "fuzzy",
             backend = "cpu")
```

Changing `backend` to `"metal"` or `"cuda"` requests that backend explicitly. If the compiled backend or required system library is absent, execution stops; the call is never silently performed on CPU and reported as GPU.

Precomputed-neighbor workflows expose the embedding boundary directly:

```
prepared_t <- prepare_opentsne_knn(indices, distances,
                                   perplexity = 15)
prepared_u <- prepare_umap_knn(indices, distances)
y_t <- opentsne_knn(prepared_t, Y_init = init, backend = "cpu")
y_u <- umap_knn(prepared_u, graph_mode = "fuzzy", backend = "cpu")
```

These calls do not recompute a KNN graph. They are useful when comparing seeds, graph modes, or CPU/GPU optimizers under the same neighborhood input. The one-call objects store total, preprocessing, KNN, initialization, and embedding timing; cross-package claims in the main paper nevertheless use total elapsed time because competing packages do not expose the same boundaries.

The PCA API is intentionally small:

```
p <- pca(x, ncomp = 2, center = TRUE, scale = FALSE,
        backend = "metal", opentsne_init = TRUE)
init <- p$opentsne_init
```

No alternative SVD selector is exposed. The returned object includes scores, loadings, singular values when available, centering/scaling vectors, backend metadata, and the seed.

S4 UMAP Mathematics and Engineering

For point i , let d_{ij} denote the distance to neighbor j . The fuzzy path estimates ρ_i from the local-connectivity rule and obtains σ_i by binary search so that the effective neighbor mass matches the requested neighborhood scale. Directed memberships are

$$w_{j|i} = \exp \left\{ -\frac{\max(0, d_{ij} - \rho_i)}{\sigma_i} \right\}.$$

For reciprocal edges, the symmetric fuzzy union is

$$w_{ij} = w_{j|i} + w_{i|j} - w_{j|i}w_{i|j}.$$

The binary mode instead assigns unit weight to every edge in the symmetric KNN union. Binary mode therefore changes graph weighting, not the neighbor identities or optimizer family, and is always recorded in the returned configuration.

The host CPU path constructs CSR `row_ptr`, `col_idx`, `weight`, and `epochs_per_sample` arrays without a dense graph. Spectral initialization uses sparse normalized-adjacency products with a connectivity-aware iteration rule. The low-dimensional optimizer samples positive edges by their schedule, generates negative vertices with a package-local RNG, applies the UMAP attraction/repulsion terms, and linearly decays the learning rate. Large default runs use 200 epochs and five negative samples; these values are selected internally rather than exposed as a broad parameter grid.

The CPU implementation parallelizes independent rows and uses contiguous float32 coordinate buffers. Metal uploads the CSR schedule once and batches unchanged epochs in command buffers. CUDA uses int32/float32 device arrays, on-device graph preparation for GPU-resident KNN, and atomic low-dimensional updates. Atomic update order can vary across runs or hardware; fixed seeds control initialization and random streams but do not promise bitwise equality across GPU executions.

S5 openTSNE-Style t-SNE Mathematics and Engineering

For each row, `fastEmbedR` searches the Gaussian precision β_i so that the entropy of

$$p_{j|i} = \frac{\exp(-\beta_i d_{ij}^2)}{\sum_k \exp(-\beta_i d_{ik}^2)}$$

matches the requested perplexity. The sparse joint affinity is the symmetrized and globally normalized matrix P . In the embedding, $q_{ij} \propto (1 + \|y_i - y_j\|^2)^{-1}$ and the optimization minimizes $\text{KL}(P\|Q)$ (van der Maaten and Hinton, 2008).

The package separates sparse positive forces from negative forces. Small diagnostic problems may use exact repulsion. Large runs use an interpolation grid: coordinates are splatted bilinearly, kernel fields are convolved by a padded two-dimensional FFT, and the fields are interpolated back to points. The optimizer then applies early exaggeration, normal optimization, adaptive gains, momentum, clipping, and centering. Automatic learning rate follows the phase-specific $n/\text{exaggeration}$ rule used by modern openTSNE/opt-SNE workflows (Poličar et al., 2024; Belkina et al., 2019).

CPU execution retains FFT plans, grids, and thread scratch. Metal stores coordinates, gains, updates, mass fields, spectra, and force fields in unified GPU memory. The validated 512-point Stockham row/column transforms use radix-4 stages and threadgroup memory; generic sizes use Cooley–Tukey kernels. CUDA retains cuFFT plans and workspace and uses CUDA Graph execution for repeated iteration groups. Only the final layout and requested diagnostics cross back to R.

S6 Float32 Data Path and Memory Accounting

The C++ numerical kernels operate on float32. For ordinary R matrices, a controlled conversion is required at the native boundary; for `float::float32` input, the package reads the float payload directly. KNN distances, graph or affinity weights, layouts, gradients, gains, and update buffers remain float32, while indices and CSR offsets use int32. If the input is float32, `fastEmbedR` returns a float32 embedding; if the input is a standard R matrix, only the final layout is converted back to double.

Peak RSS does not fall by exactly 50% because precision reduction affects only numeric payloads. R object headers, labels, int32 indices, graph structure, FFT padding, FFT spectra, search indexes, allocator fragmentation, and library workspaces remain. GPU memory accounting similarly includes both persistent buffers and peak temporary search or convolution workspace.

S7 Landmark Embedding and New Samples

`landmark.tsne()` and `landmark.umap()` implement an explicit approximation rather than silently reducing the data. A subset is selected by a deterministic projection/diversity policy or supplied by the user. The subset is embedded by the same reference function and settings as a full run. Remaining observations query the fixed landmarks, receive inverse-distance/interpolation coordinates, and may receive a regularized local affine correction estimated from high-dimensional and embedded landmark neighborhoods.

For t-SNE, query-to-reference probabilities are computed once and each query is optimized while the reference embedding is fixed, following Polícar et al. (2021). For UMAP, non-landmark rows are initialized from landmark neighbors and optionally refined with short row-restricted UMAP updates while resetting landmark coordinates to their fixed values. CPU projection is parallel by query point. Metal and CUDA provide resident interpolation/refinement paths where supported. Returned objects record landmark indices, fraction, projection K, backend, reference timing, projection timing, and refinement timing so the approximation remains visible.

S8 Benchmark Protocol

S8.1 MNIST70k main example

The main paper reports all 70,000 flattened MNIST images ($p = 784$), without a PCA-reduced input matrix. Parameters were $k = 30$, perplexity 30, seeds 4, 17, and 42, and either one or four requested CPU threads. CUDA used one NVIDIA device. `fastEmbedR` used its public one-call functions on float32 input. `Rtsne::Rtsne()` and `uwot::umap()` used standard R matrices and their internal KNN calculations; the latter was evaluated in default and `fast_sgd=TRUE` modes. Method-specific documented iteration/epoch schedules were retained, because reducing epochs to equalize runtime would change the workload exposed by the public implementation.

The archived HPC runs used Debian GNU/Linux 13, R 4.5.3, OpenBLAS 0.3.33, `fastEmbedR` 0.99.0, `uwot` 0.2.4, `Rtsne` 0.17, and `float` 0.3.3. CUDA runs used an NVIDIA L40S (compute capability 8.9) with 46,068 MiB device memory and driver 595.58.03. The runtime image selected a Conda GCC 15 toolchain and R’s portable `-O2` release policy for C++ code. CUDA dependencies and package kernels included `sm.89`. The CPU model was not written to the supplied manifest; this omission is retained as a reproducibility limitation and must be corrected in the frozen submission archive.

S8.2 Quality metrics

Quality is evaluated independently of timing. Trustworthiness measures intrusions into low-dimensional neighborhoods (Venna and Kaski, 2001); KNN preservation measures the intersection of high- and low-dimensional neighbor sets; silhouette is reported only when labels are available (Rousseeuw, 1987); and embedding-space KNN accuracy measures post hoc local label agreement. Labels are never used by unsupervised fitting. For large data, metric calculations use fixed, seed-controlled stratified samples to avoid all-pairs distance matrices.

S8.3 Extended benchmark inventory

Table S3 lists the supplied HPC archive. All datasets were evaluated where a method completed within the available resources. Claims comparing two methods use only dataset-level pairs for which both rows succeeded. Failed, unavailable, and timed-out rows remain in the generated CSV files.

Table S3: Datasets in the supplied HPC benchmark archive.

Dataset		n	p	Role and provenance
COIL20		1,440	16,384	Flattened processed object images; object identity labels
USPS		11,000	256	Handwritten digits represented as 16×16 pixels
MNIST		70,000	784	Combined train/test 28×28 digit images (LeCun et al., 1998)
Fashion-MNIST		70,000	784	Combined train/test clothing images (Xiao et al., 2017)
MetRef		873	375	Metabolomics reference samples after zero-column removal, normalization, and scaling
mass41		965,282	14	Annotated mass-cytometry benchmark from the opt-SNE release
flow18		1,000,021	11	Annotated flow-cytometry benchmark from the opt-SNE release
ImageNet features		1,281,167	1,024	Precomputed DINOv2 image features with 1,000 labels (Oquab et al., 2024)
FlowRepository	FR-	5,220,347	32	asinh-transformed mass-cytometry marker channels
FCM-ZYRM				
Tabula Muris		100,102	50	Normalized single-cell RNA-seq represented by 50 PCs (Tabula Muris Consortium, 2018)
Macosko retina		44,808	50	Mouse retinal cells represented by 50 expression PCs (Macosko et al., 2015)

S9 HPC Runtime and Quality Results

The generated table below reports representative complete-function comparisons. Each entry is the median of three seeds. Runtime includes the public function’s own KNN search, initialization, graph or affinity construction, and optimization. The full machine-readable tables additionally contain interquartile ranges, peak RAM, peak GPU-memory deltas, silhouette, Preserve@15/30/50, t-SNE KL values, stage timings, and status/error fields.

Table S4: Median total runtime and quality over three seeds for representative benchmark datasets.

Dataset	Method	Backend	Seconds	Trust.	Preserve@30	Label acc.
MNIST	fastEmbedR fuzzy UMAP	CUDA	0.99	0.927	0.490	0.964
MNIST	fastEmbedR fuzzy UMAP	CPU	50.04	0.925	0.490	0.959
MNIST	uwot fast SGD	CPU	65.34	0.929	0.491	0.962
MNIST	fastEmbedR openTSNE	CUDA	1.94	0.924	0.474	0.962
MNIST	fastEmbedR openTSNE	CPU	69.56	0.924	0.477	0.952
MNIST	FIt-SNE	CPU	116.49	0.917	0.461	0.951
MNIST	Rtsne	CPU	125.83	0.907	0.469	0.957
Fashion-MNIST	fastEmbedR fuzzy UMAP	CUDA	0.97	0.964	0.575	0.735
Fashion-MNIST	fastEmbedR fuzzy UMAP	CPU	47.72	0.966	0.581	0.728
Fashion-MNIST	uwot fast SGD	CPU	61.68	0.944	0.545	0.717
Fashion-MNIST	fastEmbedR openTSNE	CUDA	1.87	0.966	0.556	0.742
Fashion-MNIST	fastEmbedR openTSNE	CPU	69.96	0.963	0.553	0.737
Fashion-MNIST	Rtsne	CPU	101.26	0.924	0.516	0.714
Fashion-MNIST	FIt-SNE	CPU	105.28	0.931	0.517	0.734
Tabula Muris	fastEmbedR fuzzy UMAP	CUDA	0.89	0.964	0.603	0.814
Tabula Muris	fastEmbedR fuzzy UMAP	CPU	33.01	0.970	0.612	0.819
Tabula Muris	uwot fast SGD	CPU	59.05	0.982	0.640	0.827
Tabula Muris	fastEmbedR openTSNE	CUDA	3.09	0.983	0.632	0.841
Tabula Muris	fastEmbedR openTSNE	CPU	48.91	0.977	0.618	0.840
Tabula Muris	FIt-SNE	CPU	59.03	0.977	0.609	0.839
Tabula Muris	Rtsne	CPU	105.21	0.933	0.563	0.814
Retina	fastEmbedR fuzzy UMAP	CUDA	0.66	0.912	0.399	0.965
Retina	fastEmbedR fuzzy UMAP	CPU	17.93	0.919	0.407	0.964
Retina	uwot fast SGD	CPU	20.83	0.919	0.407	0.964
Retina	fastEmbedR openTSNE	CUDA	0.92	0.925	0.424	0.966
Retina	fastEmbedR openTSNE	CPU	19.01	0.925	0.419	0.968
Retina	FIt-SNE	CPU	36.72	0.924	0.411	0.968
Retina	Rtsne	CPU	52.27	0.920	0.413	0.968
flow18	fastEmbedR fuzzy UMAP	CUDA	1.77	0.965	0.427	0.952
flow18	fastEmbedR fuzzy UMAP	CPU	398.82	0.965	0.430	0.954
flow18	uwot fast SGD	CPU	789.13	0.966	0.429	0.956
flow18	fastEmbedR openTSNE	CUDA	5.47	0.977	0.441	0.956
flow18	fastEmbedR openTSNE	CPU	356.27	0.975	0.439	0.955
flow18	FIt-SNE	CPU	464.38	0.974	0.436	0.953
flow18	Rtsne	CPU	1925.53	0.777	0.226	0.745

Dataset	Method	Backend	Seconds	Trust.	Preserve@30	Label acc.
mass41	fastEmbedR fuzzy UMAP	CUDA	2.15	0.970	0.409	0.900
mass41	uwot fast SGD	CPU	407.71	0.971	0.411	0.896
mass41	fastEmbedR fuzzy UMAP	CPU	411.15	0.971	0.410	0.907
mass41	fastEmbedR openTSNE	CUDA	5.77	0.975	0.416	0.887
mass41	fastEmbedR openTSNE	CPU	339.48	0.973	0.409	0.879
mass41	FIt-SNE	CPU	403.20	0.970	0.398	0.889
mass41	Rtsne	CPU	2307.30	0.868	0.299	0.762

Across the nine datasets with successful paired rows, four-thread CPU openTSNE was a median 1.81-fold faster than full Rtsne and used 0.81 times its peak RSS; CUDA openTSNE was 54.2-fold faster than Rtsne. Median trustworthiness differences were +0.039 and +0.041. Four-thread fuzzy UMAP was 1.31-fold faster than uwot fast SGD and used 0.63 times its peak RSS; CUDA fuzzy UMAP was 63.5-fold faster. Its median trustworthiness differences were -0.0012 on CPU and CUDA. One-to-four-thread median speedups were 1.62 for openTSNE, 1.31 for fuzzy UMAP, and 1.39 for binary UMAP over nine paired datasets.

The Python benchmark branch evaluated Python openTSNE, umap-learn, and RAPIDS cuML t-SNE/UMAP both through `reticulate` and, where available, as a direct Python subprocess. Timing modes remain separate: native Python fit time is not presented as equivalent to an R end-to-end call that includes process startup, input loading, conversion, and transfer. The complete three-seed Python summary is archived as `generated/python_summary_median.csv`.

S10 Reference-Implementation Validation

The HPC validation uses a fixed 2,000-point MNIST subset to compare KNN recall, sparse affinity support/weights, and fuzzy UMAP graph weights. It also evaluates three-seed Procrustes and neighborhood stability. The purpose is behavior validation, not speed measurement or bitwise identity.

Table S5: MNIST fixed-input reference agreement from the HPC archive.

Comparison	Value	Reference
t-SNE affinity edge Jaccard	1.000000	Python openTSNE
t-SNE affinity weight Pearson	1.000000	Python openTSNE
t-SNE affinity weight Spearman	0.852039	Python openTSNE
Fuzzy UMAP graph edge Jaccard	1.000000	uwot similarity graph
Fuzzy UMAP weight Pearson	0.999990	uwot similarity graph
Fuzzy UMAP weight Spearman	0.999977	uwot similarity graph

Parallel stochastic implementations are expected to differ by rotation, reflection, scale, and update order. Validation therefore combines objective values, graph-weight agreement, Procrustes-aligned coordinates, and neighborhood measures rather than demanding identical coordinates. For full MNIST openTSNE, pairwise CPU and CUDA Procrustes correlations were at least 0.9993 and neighbor stability@15 ranged from 0.929 to 0.953. UMAP was less coordinate-stable, as expected for asynchronous stochastic graph optimization, while its neighborhood metrics remained stable.

S11 Precomputed KNN and PCA Boundaries

Embedding-from-KNN profiling caches the KNN input so that affinity/graph construction and optimization are measured without repeating neighbor search. These timings are diagnostic and are not mixed with the main end-to-end comparison.

Table S6: MNIST70k median public-function and precomputed-KNN timings.

Method	Backend	Full (s)	From KNN (s)
openTSNE	CPU, 4 threads	69.556	33.457
openTSNE	CUDA	1.938	0.618
Fuzzy UMAP	CPU, 4 threads	50.042	16.578
Fuzzy UMAP	CUDA	0.991	2.651

The CUDA fuzzy UMAP KNN-input path was slower than its one-call path in this archive, illustrating why stage timings should not be substituted for full-function comparisons: buffer residency, setup, and route selection differ across public boundaries. The PCA comparison also showed that CPU `fastEmbedR` RSVD closely matched `irlba` (MNIST Procrustes correlations 0.996–0.999) but was slower in this run (2.47 versus 1.29 seconds). CUDA PCA required 2.83 seconds. These results motivate further optimization but do not affect the embedding-quality claims.

S12 Landmark Validation

The landmark benchmark used a 50% reference subset and reported reference embedding, projection-KNN, refinement, and transform timings separately. Across 11 four-thread CPU datasets, median full/landmark speedup was 1.08 for openTSNE and 1.82 for binary UMAP; median trustworthiness changes were -0.0038 and -0.0008 . On MNIST, landmark openTSNE was slower (67.95 versus 63.15 seconds), whereas landmark binary UMAP was 1.10-fold faster (95.26 versus 105.03 seconds). CUDA landmark rows failed in the supplied run and are retained as failed statuses; no CUDA landmark claim is made from this archive.

S13 Reproducibility and Submission Freeze

The repository is <https://github.com/tkcaccia/fastEmbedR>. Each publication run should archive: the source commit and release tag; R `sessionInfo()`; resolved R CMD `config` compiler/linker commands and flags; generated `src/Makevars`; CPU model and thread environment; GPU model, driver, CUDA architecture list, CUDA host compiler, CUDA, FAISS, cuVS, RAFT, RMM, CCCL, cuFFT, Metal, and MPS versions where relevant; seed and complete method parameters; status and error text; input object dimensions and precision; and exact command lines. Benchmark rows must record the requested and actual backend independently.

The supplied benchmark manifests record package version 0.99.0 but leave the git-commit field empty. The source tree used to revise this manuscript was at repository HEAD `12a4a701b5296eb69418c707c603e64d810d1fc8` with additional approved working-tree changes. The final submission must therefore freeze a clean release commit, rerun or bind the archive to that commit, and add its Zenodo DOI. The LaTeX source uses the unmodified JMLR style file at upstream commit `f413f638b407af76074813f8f88a82a7a5a81e9d`.

References

- Anna C. Belkina, Christopher O. Ciccolella, Rina Anno, Richard Halpert, Josef Spidlen, and Jennifer E. Snyder-Cappione. Automated optimized parameters for t-distributed stochastic neighbor embedding improve visualization and analysis of large datasets. *Nature Communications*, 10:5415, 2019. doi: 10.1038/s41467-019-13055-y.
- Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. doi: 10.1109/5.726791.
- Evan Z. Macosko, Anindita Basu, Rahul Satija, James Nemesh, Karthik Shekhar, Melissa Goldman, Itay Tirosh, Allison R. Bialas, Nolan Kamitaki, Emily M. Martersteck, John J. Trombetta, David A. Weitz, Joshua R. Sanes, Alex K. Shalek, Aviv Regev, and Steven A. McCarroll. Highly parallel genome-wide expression profiling of individual cells using nanoliter droplets. *Cell*, 161(5):1202–1214, 2015. doi: 10.1016/j.cell.2015.05.002.
- Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khilodov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mahmoud Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jégou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. DINOv2: Learning robust visual features without supervision. *Transactions on Machine Learning Research*, 2024. URL <https://openreview.net/forum?id=a68SUt6zFt>.
- Pavlin G. Poličar, Martin Stražar, and Blaž Zupan. Embedding to reference t-SNE space addresses batch effects in single-cell classification. *Machine Learning*, 110:1211–1231, 2021. doi: 10.1007/s10994-021-06043-1.
- Pavlin G. Poličar, Martin Stražar, and Blaž Zupan. openTSNE: A modular python library for t-SNE dimensionality reduction and embedding. *Journal of Statistical Software*, 109(3):1–30, 2024. doi: 10.18637/jss.v109.i03.
- Peter J. Rousseeuw. Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20:53–65, 1987. doi: 10.1016/0377-0427(87)90125-7.
- Tabula Muris Consortium. Single-cell transcriptomics of 20 mouse organs creates a Tabula Muris. *Nature*, 562:367–372, 2018. doi: 10.1038/s41586-018-0590-4.
- Laurens van der Maaten and Geoffrey Hinton. Visualizing data using t-SNE. *Journal of Machine Learning Research*, 9:2579–2605, 2008. URL <https://www.jmlr.org/papers/v9/vandermaaten08a.html>.
- Jarkko Venna and Samuel Kaski. Neighborhood preservation in nonlinear projection methods: An experimental study. In *Artificial Neural Networks – ICANN 2001*, pages 485–491. Springer, 2001. doi: 10.1007/3-540-44668-0_68.

Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017. doi: 10.48550/arXiv.1708.07747.